

Refonte du site web

VINDHELLFEST



ARNAUD Aurélien

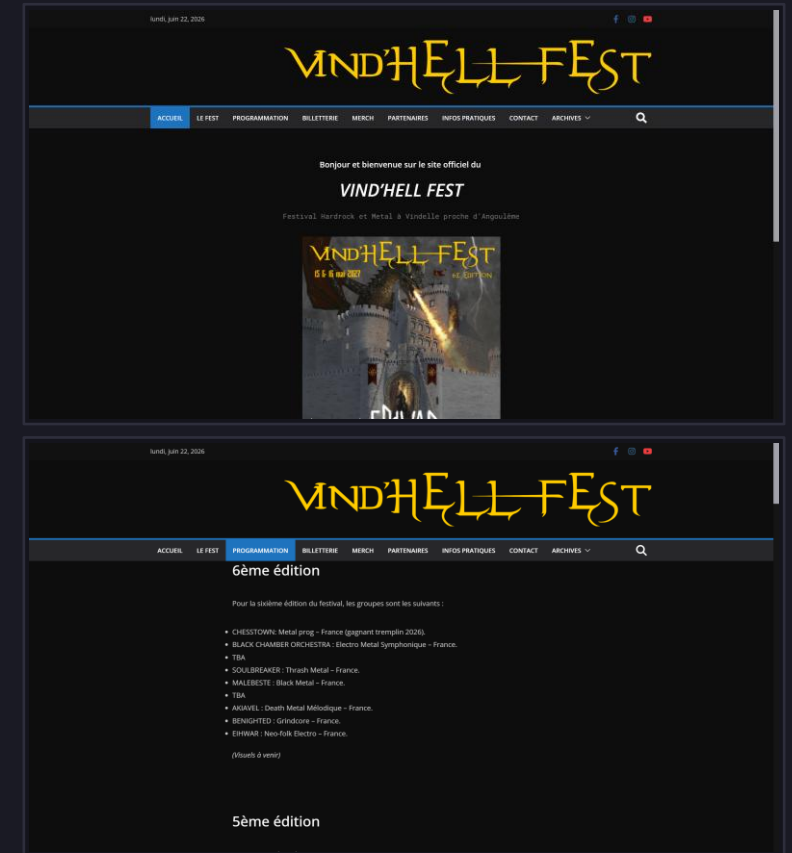
Soutenance CDA — Concepteur Développeur d'Applications · Juillet 2026

Plan de la présentation

- | | | | |
|----------|----------------------------------|-----------|---------------------|
| 1 | Contexte & besoins | 6 | Focus code |
| 2 | Gestion de projet | 7 | Sécurité |
| 3 | Conception (UML, maquettes, BDD) | 8 | Tests & déploiement |
| 4 | Architecture technique | 9 | Veille |
| 5 | Démonstration live | 10 | Bilan |

Le problème

- Site sous WordPress / PHP, design vieillissant et statique
- Aucune page d'actualités → communication 1 à 2 fois par an
- Chaque ajout d'artiste ou d'actu passait par un prestataire externe → lenteur, dépendance, coût



Ancien site — vindhellfest.fr (WordPress) : accueil & programmation

Objectifs & priorisation

5 OBJECTIFS

- Modernisation de l'image
- Vitrine publique
- Administration autonome
- Gestion différenciée des droits
- Qualité & maintenabilité

PRIORISATION MOSCOW

Must-have

- Vitrine publique
- Administration autonome
- Qualité & maintenabilité
- Accessibilité (WAVE : 0 erreur, 0 contraste)

Should-have (livrable sans)

- Archives des éditions passées
- Statistiques de fréquentation

Périmètre & rôles

In scope

Site public · admin sécurisé (CRUD artistes / actus / utilisateurs) · upload d’images · e-mails transactionnels · CI/CD

Out of scope

Billetterie (HelloAsso) · boutique merchandising · multilingue (site en français)

4 profils : Visiteur public · Admin (accès complet + comptes) · Gestionnaire artists · Gestionnaire news

Fonctionnalité	admin	artists	news
Artistes (CRUD + upload image)	✓	✓	—
Mise en avant (is_featured)	✓	✓	—
Actualités (CRUD + brouillon/publié)	✓	—	✓
Utilisateurs (CRUD + attribution de rôle)	✓	—	—
Changement de mot de passe	✓	✓	✓

Méthode & organisation

- Projet mené en autonomie, du cahier des charges à la mise en production
- 4 sous-projets interdépendants : infra Docker + CI/CD → BDD, backend, frontend
- Back et front en parallèle, chacun avec sa doc et ses tests
- Backlog kanban à 5 états (À faire → Terminé, + Bloqué)

The screenshot shows a Notion backlog card titled "Connexion d'un utilisateur". It includes a complexity level of 3, a functional layer of "Authentification", and technical layers for "Backend" and "Frontend". The priority is "P1" and the state is "Terminé". The user story describes a user logging in with email and password. Below this, there are acceptance criteria and technical notes. The acceptance criteria list requirements for the login form, submission button, error messages, redirection, forgot password button, and email reset. The technical notes specify the page location, endpoints for login and forgot password, the backend controller, the sessions table, and the cookie configuration for JWT.

Connexion d'un utilisateur

Complexité 3

📁 Couche fonction... **Authentification**

📁 Couche technique **Backend** **Frontend**

📌 Priorité **P1**

📖 User Story En tant qu'utilisateur inscrit, je veux me connecter avec mon email et mon mot de passe afin d'accéder à mon espace personnel.

📌 État **Terminé**

+ Add a property

Commentaires

Ajouter un commentaire...

Critères d'acceptation

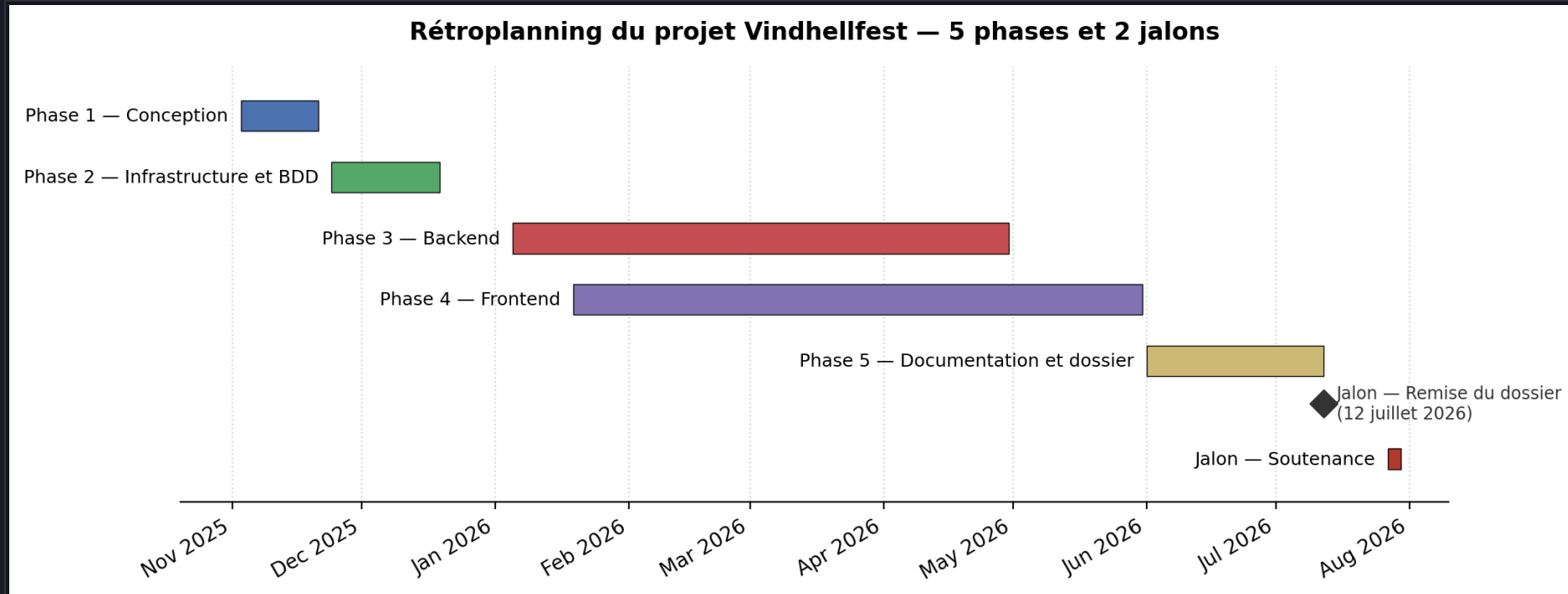
- ✓ Formulaire avec champs **email** et mot de passe (labels **se-only** pour l'accessibilité)
- ✓ Bouton de soumission désactivé si un champ est vide ou si la requête est en cours
- ✓ Message d'erreur générique affiché sous le formulaire en cas d'échec
- ✓ Redirection vers **/admin/dashboard** après connexion réussie
- ✓ Bouton "Mot de passe oublié" ouvrant une modale dédiée
- ✓ La modale "Mot de passe oublié" envoie un nouveau mot de passe par **email**
- ✓ Cookie **httpOnly JWT** créé et session enregistrée en base après connexion

Notes techniques

- Page : **src/app/auth/login/page.tsx** — composant client
- **Endpoint** connexion : **POST /admin/auth/login**
- **Endpoint** mot de passe oublié : **POST /admin/auth/forgot-password**
- **Backend** : **controllers/admin/auth/login.controller.ts**
- Session : table **sessions** (user_id, expires_at)
- Cookie : **httpOnly** configuré via **COOKIE_ACCESS_TOKEN** dans **.env.backend**
- Modale : composant **ForgotPassword** via **react-modal**

Exemple de carte backlog (Notion)

Rétroplanning & outils

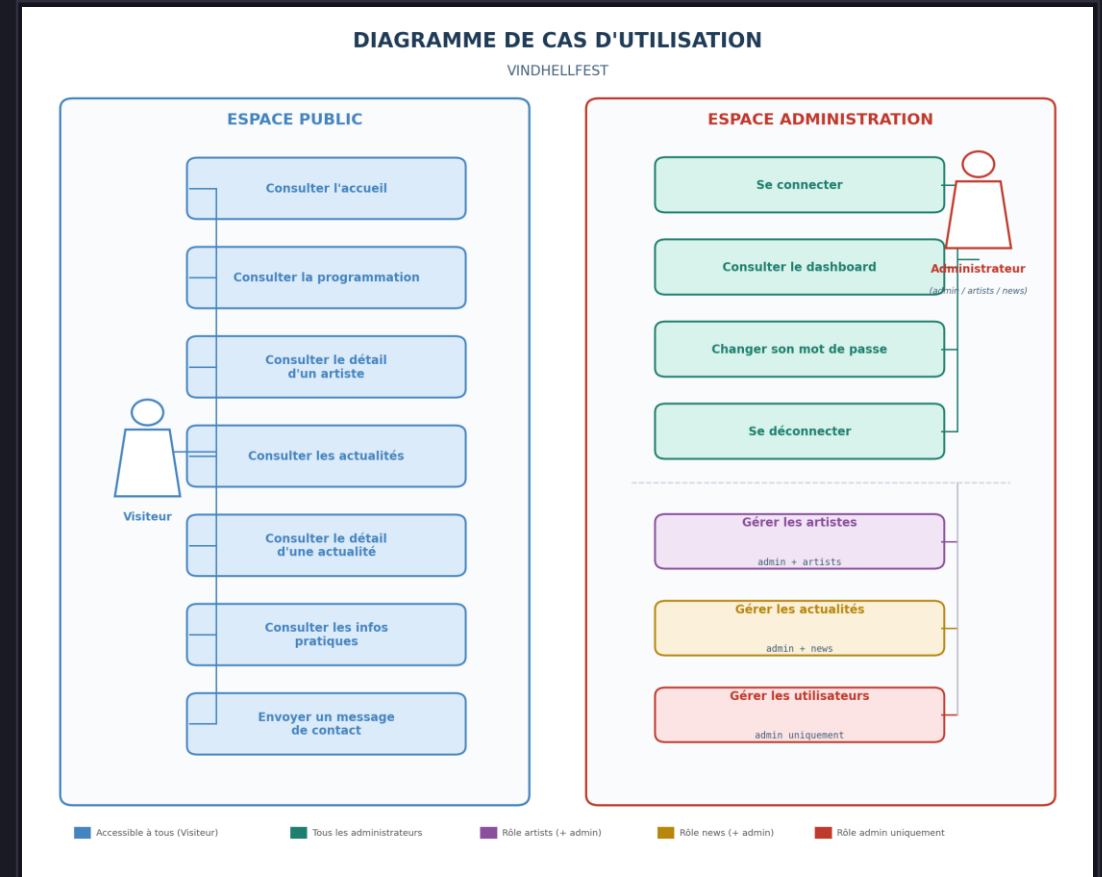


Rétroplanning — 5 phases (backend & frontend chevauchés) et 2 jalons

- Jalons : remise du dossier 12 juil. · soutenance 27-30 juil. 2026
- Outils : Git/GitHub · GitHub Actions (2 jobs) · Docker Compose · draw.io / pgAdmin · Figma · VS Code

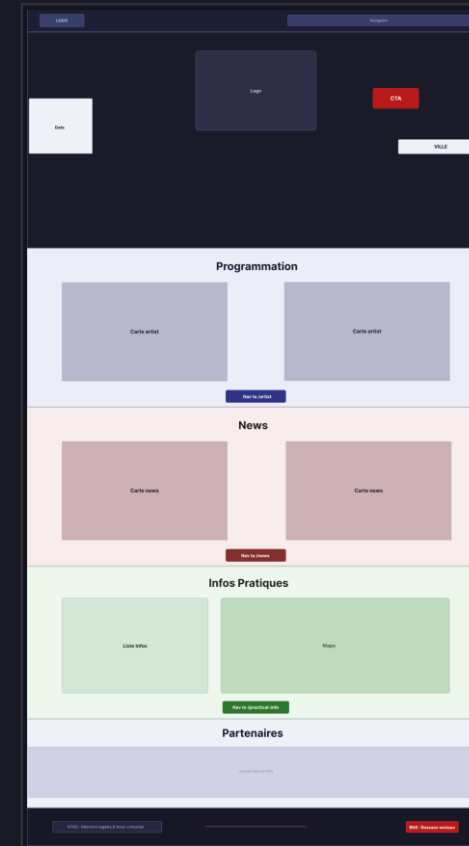
Cas d'utilisation

- 2 acteurs : Visiteur (sans authentification) · Administrateur (admin / artists / news)
- Visiteur — 7 cas publics : accueil, programmation, programmation, artiste, actualités, infos, contact contact
- Admin — 4 cas communs (connexion, dashboard, mot de passe, déconnexion)
- + 3 cas restreints : artistes (admin+artists), actus (admin+news), utilisateurs (admin)

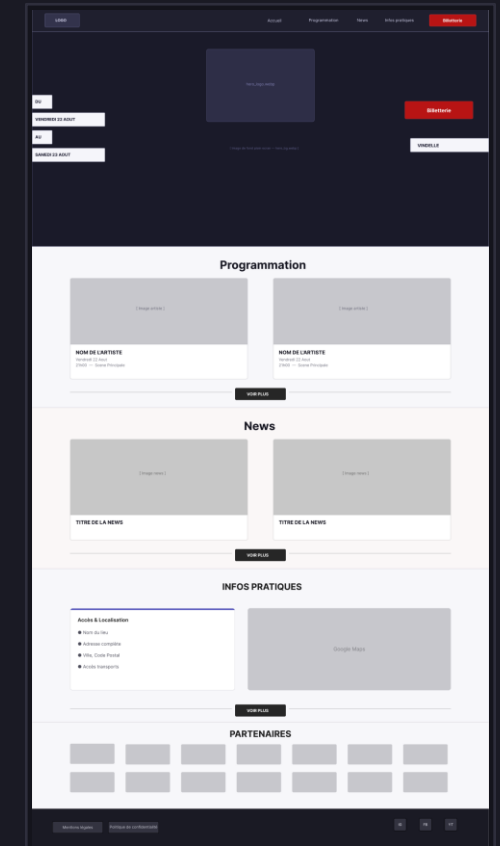


Maquettes & design

- Démarche Figma : zoning → wireframe basse fidélité → maquette
- 13 zonings + 13 wireframes couvrant tous les écrans (public + admin)
- Structure validée avant la charte graphique
- Responsive desktop / tablette / mobile · 2 thèmes (visitor sombre, admin clair)



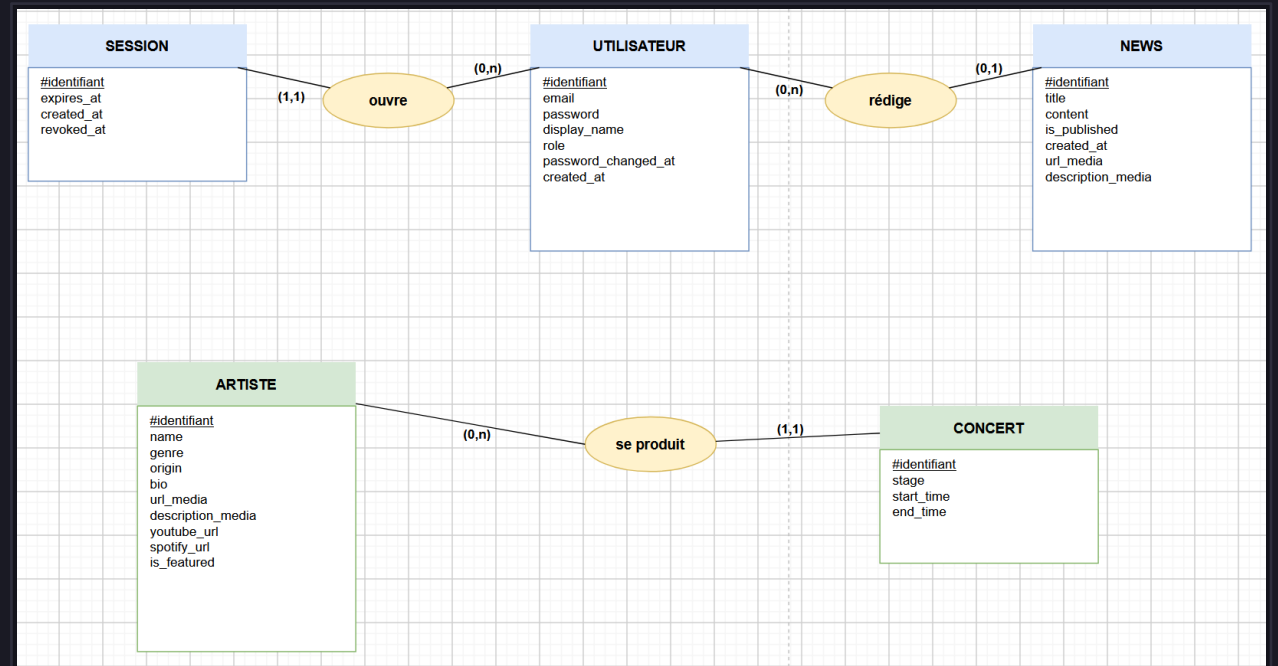
Zoning



Wireframe

Modélisation de la base (MCD → MLD → MPD)

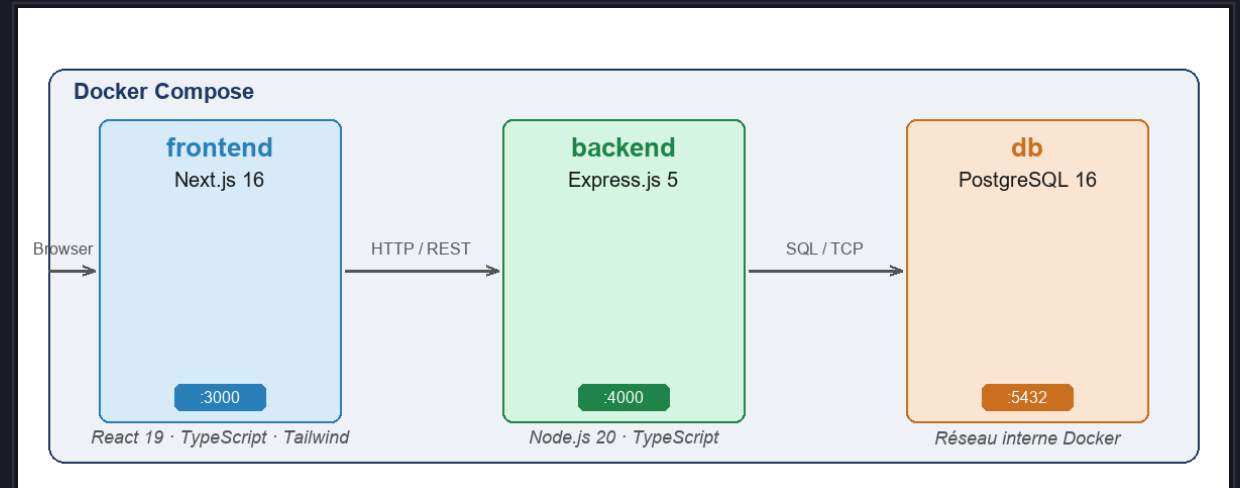
- Démarche Merise : MCD → MLD → MPD
- 5 entités : Utilisateur, Session, News, Artiste, Concert
- Types PostgreSQL : UUID, CITEXT, VARCHAR, TEXT, BOOLEAN, TIMESTAMPTZ
- Règles métier en base : CHECK, EXCLUDE (anti-chevauchement), trigger « max 2 en avant »



MCD — 5 entités et leurs relations

Vue d'ensemble de l'architecture

- Architecture 3 tiers orchestrée par Docker Compose :
Next.js (3000) ↔ API Express (4000) ↔ PostgreSQL (5432)
- Conteneurs isolés sur un réseau Docker interne (app-net)
- API REST sans état, requêtes admin par cookie JWT
httpOnly vérifié en base
- Double URL d'appel API : côté conteneur vs côté navigateur



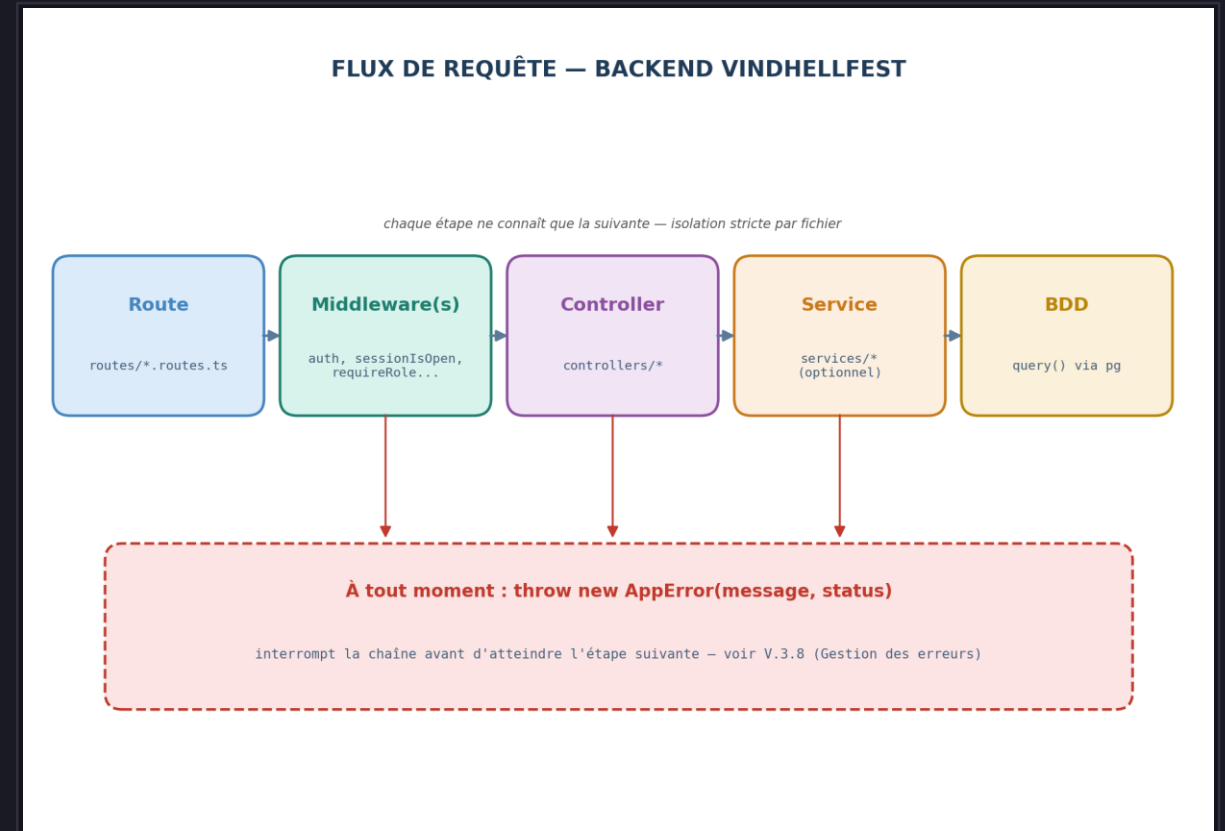
Architecture 3 tiers — Docker Compose

Stack & justification des choix

- **Next.js 16 / React 19** → SEO d'un site grand public (SSR/SSG/ISR)
- **Express 5 / Node 20** → architecture en couches maîtrisée, TS strict
- **PostgreSQL 16** → citext, btree_gist (contraintes EXCLUDE)
- **JWT + cookies httpOnly + sessions** → anti-XSS, révocation possible
- **Zod** → validation des entrées à l'exécution
- **Vitest + Supertest + RTL** → tests rapides, sans port réseau réel
- **Docker Compose** → environnement identique dev / CI

Architecture en couches (backend)

- Backend structuré en couches : routes → middlewares → contrôleurs → services → BDD
- Chaque couche ne connaît que la suivante : un fichier par responsabilité
- Pas de MVC monolithique : front et back séparés, la Vue React interroge l'API en asynchrone



Flux d'une requête à travers les couches

LIVE

Démonstration

Espace public

Accueil → Programmation → Fiche artiste → Actualités → Infos pratiques → Contact

Back-office administration

Connexion → Dashboard → CRUD artiste (+ upload) → Publier une actualité → Gestion des utilisateurs

Focus code — règle métier en base

CODE EN DIRECT

```
bd/init/05_concert_schema.sql · 04_artist_schema.sql ·  
create_artist.controller.ts
```

- Contrainte EXCLUDE USING gist → aucun chevauchement de concerts (garanti en base, pas en JS)
- Trigger check_featured_limit → maximum 2 artistes mis en avant
- Exception PostgreSQL interceptée dans le catch → traduite en erreur métier 409 (+ ROLLBACK)

Focus code — couche d'accès aux données

CODE EN DIRECT

`apps/frontend/src/functions/apiRequest.ts`

- Union discriminée { data | error } → traitement d'erreur forcé par TypeScript, pas de try/catch dans les composants
- Cookie httpOnly envoyé automatiquement via credentials: "include"
- Gestion d'erreur centralisée : 3 cas (erreur API, erreur technique, cas limite)

Authentification & contrôle d'accès

- **JWT signé** — payload minimal (userId, sessionId), 1h
- **Cookie httpOnly** — anti-XSS, + Secure, SameSite=lax
- **Sessions en base** — révocation explicite (logout, cascade)
- **RBAC serveur** — requireRole → 403 ; frontend = confort



Défense en profondeur

- **Validation Zod (backend)** — verrou réel des entrées → 400
- **Injection SQL** — requêtes paramétrées, zéro concaténation
- **Rate limiting** — 5 essais / 10 min / IP → 429
- **Upload sécurisé** — MIME, 5 Mo, WebP, renommage UUID
- **Hachage bcrypt** — 10 rounds, sel auto ; jamais en clair
- **CORS** — restreint à une seule origine + credentials
- **Secrets hors code** — .env + validation fail-fast (19 variables)

Stratégie de tests & jeu d'essai

PYRAMIDE DE TESTS



241

tests passés

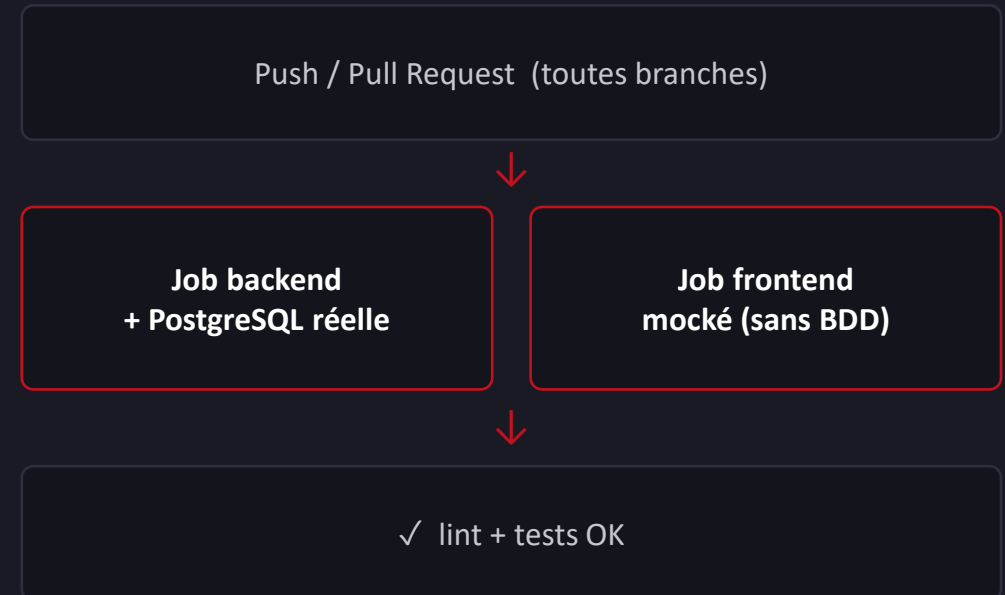
backend 132 · frontend 109 · 39 fichiers · aucune régression

JEU D'ESSAI — POST /admin/artists

Cas	Attendu	Obtenu
Nominal (multipart)	HTTP 201	201 ✓
Image manquante	HTTP 400	400 ✓

Déploiement & DevOps

- **Docker Compose** — 3 services, Dockerfiles multi-stages (dev → runner)
- **CI GitHub Actions** — 2 jobs parallèles, lint + tests à chaque push / chaque push / PR
- **Reproductible** — environnement identique dev / CI
- **Déploiement prod** — pas encore automatisé (CD runner envisagé)



Veille techno & sécurité

VEILLE APPLIQUÉE — 2 CAS VÉCUS

✓ Vindhellfest Veille sécurité	44
 CERT-FR	43
 Node.js Blog: Vulnerability Reports	1
✓ Vindhellfest Veille technologique	72
 Archive: 2026 - GitHub Changelog	... 34
 Docker	9
 Next.js Blog	3
 Node.js Blog	8
 PostgreSQL news	18
 React Blog	

Feedly — 2 dossiers : sécurité & technologique

Hot reload en Docker

Issue GitHub officielle Next.js #80665 : Turbopack ne respecte pas le polling en conteneur. → Contournement documenté retenu : retour au moteur Webpack.

CVE PostCSS (CVE-2026-41305)

Signalée par npm audit (dépendance embarquée par Next.js). Analyse : PostCSS ne traite que nos CSS internes à la compilation → scénario d'attaque inapplicable. Décision motivée : ne pas patcher (le fix imposerait un Next.js très ancien) et surveiller une version corrigée.

Bilan & difficultés surmontées

Chevauchement des concerts

→ Réglé en base : extension btree_gist + contraintes EXCLUDE USING gist. Fiable même en accès concurrent.

Hot reload en Docker

→ Turbopack ne respecte pas le polling en conteneur. Retour au moteur Webpack (NEXT_DISABLE_TURBOPACK=1).

Double URL d'API

→ Le conteneur et le navigateur n'adressent pas le backend pareil. Deux variables : API_URL_SERVER (SSR) et NEXT_PUBLIC_API_URL (navigateur).

Compétences couvertes & perspectives

AT1 — DÉVELOPPER UNE APPLICATION SÉCURISÉE

Environnement · interfaces · composants métier · gestion de projet

AT2 — CONCEVOIR ET DÉVELOPPER EN COUCHES

Besoins & maquettes · architecture · base de données · accès aux données

AT3 — PRÉPARER LE DÉPLOIEMENT

Plans de tests · déploiement documenté · mise en production DevOps

PERSPECTIVES

- Hébergement en ligne + CD automatisé
- Stockage objet externe pour les images
- Boutique de merchandising
- Archives des éditions passées
- Statistiques de fréquentation

11 / 11 compétences du référentiel

Merci de votre attention

Je suis à votre disposition pour vos questions

Vindhellfest — Soutenance CDA · ARNAUD Aurélien